closure-evidence

BOB-139 — SSE disconnect probe fail-open → fail-closed

Revision: 1 **Last modified:** 2026-08-20T15:49:48Z **Status:** Fixed (→ pending §11.4.120 reconciliation of 2 stale gates — see §6) **Type:** Bug **Track/label:** (T1/main - claude4 - opus - xhigh)

---

1. The defect

Both SSE generators in `download-proxy/src/api/streaming.py` guarded their `while True` loop with a byte-identical closure:

```
async def _client_gone() -> bool:
    if request is None:
        return False
    try:
        return await request.is_disconnected()
    except Exception:
        return False
```

The bare `except Exception: return False` meant a raising disconnect probe resolved to “client is still connected”, so the generator streamed forever.

- **§11.4.252 fail-closed-on-dangerous-combination** — the probe is the *only* condition that terminates the loop. A path that cannot verify its precondition must refuse, not proceed.
- **§11.4.201(6) false-null** — a raising probe and a genuinely-connected client returned the identical `False`; the loop could not distinguish “the client is here” from “I am blind”.

Blast radius found during investigation (not in the original brief)

`download-proxy/src/api/routes.py:773-780` wraps `search_results_stream` in a `_sse_stream_count` guard capped by `_SSE_STREAM_MAX` (default **32**, line 720) and decrements it in a `finally`. A generator that never returns never runs that `finally`, so the counter never decrements. **After 32 leaked streams every subsequent client receives HTTP 429** and the SSE feature is dead for everyone — the leak is not merely untidy, it is a self-inflicted denial of the feature.

2. The fix

The two byte-identical closures were replaced by ONE module-level helper (§11.4.251 — two near-identical forks differing only in an id variable are exactly the duplication that drifts):

```
async def _stream_stop_reason(request: Request | None, stream_id:
        str) -> str | None
```

Returns a close reason when the loop MUST stop, `None` when it may continue.

Why this specific catch shape

Branch	Behaviour	Why
`request is None`	`return None` (continue)	Nothing was probed, so nothing is *indeterminate* — the caller opted out of disconnect detection. Failing closed here would kill every request-less caller on iteration 1.
`except asyncio.CancelledError: raise`	propagate	Task teardown, **not** a probe

Branch	Behaviour	Why
		failure. Converting cancellation into an ordinary close breaks structured cancellation — a different defect. On Python 3.8+ `CancelledError` derives from `BaseException` and already escapes `except Exception`; naming it explicitly makes that guarantee **load-bearing rather than incidental**, and keeps it true if the catch is ever widened.
`except Exception`	log WARNING, `return CLOSE_REASON_PROBE_FAILED`	Fail closed + **surface the fault**. A silent fail-closed beats a fail-open but still hides a real error, so the log carries the stream id, exception type and message.
probe returns truthy		

Branch	Behaviour	Why
	`return CLOSE_REASON_CLIENT_DISCONNECTED`	Unchanged genuine-disconnect path.

Why a *distinct* close reason

A probe failure emits `reason: "disconnect_probe_failed"`, not `"client_disconnected"`. We do **not** know the client disconnected — only that we cannot see. Reporting the second as the first would be a §11.4.6 misstatement inside the event stream itself, and it makes the fault greppable in the stream, not only in logs. The genuine-disconnect reason string is unchanged, so no existing consumer contract is broken (verified: only `tests/unit/test_streaming.py` and a historical migration doc reference the literal).

Why fail-closed is the safe direction (the asymmetry)

SSE clients reconnect by design (`EventSource` retry), so a spurious close costs **one reconnect and is fully recoverable**. A fail-open costs a leaked socket and task for the lifetime of the process, is **invisible**, and exhausts the stream cap. A visible, recoverable stop beats an invisible, unbounded leak (§11.4.101).

3. RED → GREEN, both polarities, both generators

New test: `tests/unit/api_layer/test_sse_client_gone.py` (12 tests), written and run **before** the fix. A fail-open loop never ends, so rather than hanging on a wall-clock timeout every stub counts its own polls and raises `_StreamDidNotStop` once the budget is spent — deterministic (§11.4.50), no wall clock.

RED (pre-fix) — first failure names the defect directly

```
E   api_layer.test_sse_client_gone._StreamDidNotStop:
search_results_stream kept
    polling for 51 iterations — it never stopped (fail-OPEN)
tests/unit/api_layer/test_sse_client_gone.py:130:
```

```
_StreamDidNotStop
1 failed in 2.17s
```

Full pre-fix run — **4 defect-capturing tests FAIL, 8 controls PASS**:

```
PASSED  test_search_stream_continues_while_client_connected
PASSED  test_search_stream_genuine_disconnect_keeps_its_own_reason
PASSED  test_search_stream_propagates_cancellation
PASSED  test_search_stream_without_request_is_not_failed_closed
PASSED  test_download_stream_continues_while_client_connected
PASSED
test_download_stream_genuine_disconnect_keeps_its_own_reason
PASSED  test_download_stream_propagates_cancellation
PASSED  test_download_stream_without_request_is_not_failed_closed
FAILED  test_search_stream_terminates_when_probe_raises
FAILED  test_search_stream_logs_probe_failure
FAILED  test_download_stream_terminates_when_probe_raises
FAILED  test_download_stream_logs_probe_failure
4 failed, 8 passed in 5.72s
```

Honest note (§11.4.6): the 8 control tests pass in *both* polarities by design. That is their job — they exist to catch a fix that over-terminates, so they are regression guards, not RED-capturing evidence. Only the 4 listed failures capture the escape. This is stated rather than presented as "12 RED".

GREEN (post-fix) — all 12, controls still green

```
WARNING  api.streaming:streaming.py:95 SSE disconnect probe failed
for sid-raise (RuntimeError: receive channel gone) - closing
stream fail-closed
WARNING  api.streaming:streaming.py:95 SSE disconnect probe failed
for sid-log (RuntimeError: receive channel gone) - closing stream
fail-closed
WARNING  api.streaming:streaming.py:95 SSE disconnect probe failed
for dl-raise (RuntimeError: receive channel gone) - closing stream
fail-closed
WARNING  api.streaming:streaming.py:95 SSE disconnect probe failed
for dl-log (RuntimeError: receive channel gone) - closing stream
fail-closed
PASSED  test_search_stream_terminates_when_probe_raises
PASSED  test_search_stream_continues_while_client_connected
PASSED  test_search_stream_genuine_disconnect_keeps_its_own_reason
PASSED  test_search_stream_propagates_cancellation
PASSED  test_search_stream_without_request_is_not_failed_closed
PASSED  test_search_stream_logs_probe_failure
PASSED  test_download_stream_terminates_when_probe_raises
PASSED  test_download_stream_continues_while_client_connected
PASSED
test_download_stream_genuine_disconnect_keeps_its_own_reason
```

```
PASSED  test_download_stream_propagates_cancellation
PASSED  test_download_stream_without_request_is_not_failed_closed
PASSED  test_download_stream_logs_probe_failure
12 passed in 3.37s
```

Both polarities are asserted for **both** generators (§11.4.201(1)): raising probe → stream **terminates**; connected client → stream **continues uninterrupted** and runs to `search_complete` / `download_complete` with **no** `event: close` emitted.

4. Mutation battery (§1.1) — the assertions are load-bearing

Each mutation was applied to the fixed source, the suite re-run, and the source restored (verified byte-identical afterwards).

```
=== M1: fail-open restored (probe failure -> keep streaming) ===
FAILED test_search_stream_terminates_when_probe_raises
FAILED test_search_stream_logs_probe_failure
FAILED test_download_stream_terminates_when_probe_raises
FAILED test_download_stream_logs_probe_failure
4 failed, 8 passed

=== M2: over-terminate (kill every request-less stream) ===
FAILED test_search_stream_without_request_is_not_failed_closed
FAILED test_download_stream_without_request_is_not_failed_closed
2 failed, 10 passed

=== M3: cancellation swallowed ===
FAILED test_search_stream_propagates_cancellation
FAILED test_download_stream_propagates_cancellation
2 failed, 10 passed

streaming.py restored to fixed state
```

M1 is the canonical revert (§11.4.115(F)). **M2 is a mutation not written into the fix** (§11.4.194(6)(d)) — it proves the *control* polarity is real: a fix that terminated every stream would be caught, so the controls are not decoration. **M3** proves cancellation handling is asserted, not assumed.

5. Regression

```
tests/unit/api_layer/                    211 passed in 188.20s
```

Wider `*sse*` / `*stream*` unit sweep (10 files):

```
FAILED tests/unit/
test_streaming.py::TestSSEHandler::test_download_progress_stream_cl
FAILED tests/unit/
test_streaming.py::TestSearchResultsStreamEdgeCases::test_search_re
2 failed, 75 passed, 1 warning in 18.21s
```

Lint: `ruff check download-proxy/src/api/streaming.py` → **All checks passed**. (`ruff format` reports one deviation in `_build_merged_update` at line ~146 that is **pre-existing at HEAD** — verified against `git show HEAD:...` — and was not introduced or "fixed" here.)

Integration files `tests/integration/test_realtime_streaming.py` and `test_streaming_browser.py` were **NOT** run: they require the live stack, and service restarts are owned by another agent this round (§11.4.119). **Status against those: UNKNOWN — not measured, not claimed.**

6. ⚠ BLOCKING follow-up — 2 stale gates (§11.4.120), NOT fixed here

The 2 regression failures above are **not** caused by a defect in the fix. Both tests literally assert the fail-open, in their own docstrings and assertions:

```
tests/unit/test_streaming.py:105  """When request.is_disconnected
raises during download, stream continues."""
tests/unit/test_streaming.py:194  """When request.is_disconnected
raises, _client_gone returns False and stream continues."""

>   assert not any("close" in e for e in events)
E   assert not True
```

§11.4.102 investigation performed first, as §11.4.120 requires: these gates asserted **old-correct-now-removed** behaviour; they are not catching a regression the fix introduced. §11.4.120 therefore mandates **reconciliation** — rewrite the gate to assert the NEW mechanism. Both alternatives are **forbidden**: fake-passing the gate, and reverting the correct fix.

`tests/unit/test_streaming.py` is **outside this agent's assigned file ownership**, so it was deliberately left untouched (§11.4.119 single-owner — editing an unowned file to "help" is exactly the contention

that corrupts parallel work). To make the block park the edit and not the work (§11.4.101), a **self-verifying, ready-to-apply** reconciliation ships alongside this doc:

`docs/qa/BOB-139/reconcile_stale_gates.py`

Verified on a scratchpad copy (the real file untouched): applies cleanly, replaces both stale gates, leaves **0** remaining `assert not any("close" ...)` assertions and **2** `disconnect_probe_failed` assertions, and **refuses** (exit 1) on a second run so it cannot half-apply or corrupt.

The file's owner should run:

```
.venv/bin/python docs/qa/BOB-139/reconcile_stale_gates.py
nice -n 19 .venv/bin/python -m pytest tests/unit/
        test_streaming.py -q --import-mode=importlib
```

Until then the suite is knowingly RED on those 2 tests.

7. Found, honestly NOT fixed (§11.4.6)

1. **`download-proxy/src/api/routes.py:167`** — `/theme/stream` calls `await request.is_disconnected()` with **no** guard at all. That is fail-closed *in effect* (an exception ends the generator and its `finally: store.unsubscribe(queue)` still runs, so it does not leak), but it terminates via an uncaught traceback rather than a clean `event: close`. Inconsistent with the contract established here. Not my file — not touched.
2. **`theme_state.py`** / **`main.py`** — read-only grep confirms neither contains an `is_disconnected` call, so the fail-open pattern does **not** exist there. No edit was needed or made.
3. **Production probe-failure rate: UNKNOWN.** I did not measure how often `request.is_disconnected()` actually raises in this deployment, and I did not independently verify the 7 CLOSE_WAIT sockets cited in the brief. Supporting (but not conclusive) evidence that the probe is not *systematically* broken: `/theme/stream` calls it unguarded in a `while True` and that endpoint works, which a systematically-raising probe would prevent. That is inference from a working endpoint, **not** a measurement — stated as such.
4. **Pre-existing `ruff format` deviation** in `_build_merged_update` (streaming.py ~line 146) left as found; reformatting it would have mixed an unrelated change into this fix.